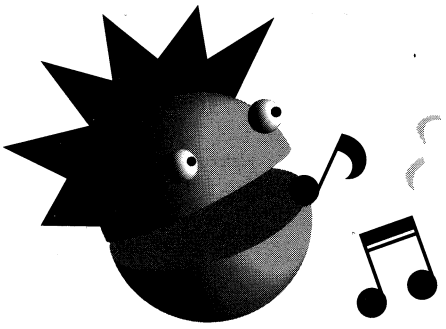


グラフィック&サウンド

PLAY3 Ver1.00

●Ryu

PJ'93年8月号の松樹さんの「ライブで行こう！」で予告した“ポケコン単体で和音演奏をしてしまうプログラム”を完成させました．

もちろんPC-E650にも対応しています．

動作環境

すべてのシャープPC-E500/1480Uシリーズで動作すると思います．

実際には，以下の機種で動作確認しました．

●PC-E500：ROM Version 3
●PC-1490U：ROM Version 6
●PC-E650：ROM Version 8

プログラムの入力方法

まず，マシン語エリアを4Kバイト以上確保します．

4Kバイト確保するには，

```
POKE &BFE03,&1A,&FD,&B,0,&10,0 ↵
CALL &FFFD8 ↵
```

とします．

次に，BASICリストをそのまま入力するか，BASICリスト中の**DATA**文のマシン語データを直接マシン語モニタ（SIMONなど）で入力してください．

BASICリストの入力が終わったら，RUN MODEで，

```
RUN ↵
```

として，プログラムを実行してください．

＊

実行すると，プログラム（マシン語）をメモリに書き込みます．

書き込んでいる途中で「**Error**」と表示されたら，画面上に表示されている行番号のデータを見直して，修正してください．

「**Completed**」と表示されれば，終了です．

＊

最後に，

```
SAVE"E:PLAY3",&BF000,&BF54F ↵
```

として，RAMファイルなどにセーブしておきます．

できればカセット・テープやフロッピー・ディスクにセーブするか，PLINKなどを利用して，パソコンに転送して保存しおくとよいでしょう．

起動方法

入力が終わったら，

```
CALL &BF000 ↵
```

として，マシン語プログラムを実行します．

実行すると「**PLAY**」「**EXOFF**」という2つのBASIC命令が拡張されます．

これでPLAYコマンドが使えるようになります．

また，**EXOFF**命令を実行すると，PLAYがBASICから切り離されますが，このときに，必ずSyntax errorが発生します．これは，故意にエラーを出しているので，安心してください．

注意！

PLAY命令を組み込んだら，①マシン語エリアの開放，②他のマシン語プログラムのロード/実行，などはできるだけしないでください．

特に，マシン語エリアの開放は絶対に行わないでください．

これは，上記の操作では，BASIC命令の拡張データを消してしまうことがあり，最悪の場合，暴走してしまう可能性があるためです．

このようなことを行ないたい場合は，必ず**EXOFF**命令で**PLAY**命令を切り離してからにしてください．

もし，**EXOFF**命令を実行せずにマシン語の開放，他のマシン語プログラムのロードなどをして動作がおかしくなったときは，リセット・スイッチを押してください．

使い方

PLAY命令が組み込まれても，命令の使い方が分からなくては意味がありません．

一般的に，**PLAY**命令は，

▼先日，「JRなど鉄道忘れ物」バザールがあるというので行ってみました．
鉄道の忘れものということなので，もしやポケコンや電子手帳があるのでは？　と思って行ってみました．
行ってみると，電子手帳こそありませんでしたが（昼ごろ行ったので売り切れていたのかも知れません），ポケコンは3台ありました．

```
PLAY"第1パート：第2パート：第3パート"
```

というように，3つのパート（音）を「：」（コロン）で区切ります．

たとえば，

```
PLAY"O2L5C：E：G"
```

とすれば，「ドミソの和音」が出ます．

使用の制限など

★パートは必ず3つ必要というわけではなく，1つでも2つでも大丈夫です．

★MMLデータは**PLAY**命令の後を直接読んでいるので，変数などは使えません．

変数の対応は，今後のバージョン・アップで対応したいと思っています．

★演奏中，停止させたいときは ON キーをしばらく押しつづけてください．

これは音の発生中は割り込みを禁止していて， BRAKE は音が変わるときだけにチェックされるためです．

★マルチ・ステートメントで記述できますが，できる限り差し控えてください．

★動作クロックは標準のクロック（2.3MHz）のみ対応です．4.8MHzなどでは音が高く，短くなります．

コマンドの説明

PLAY命令の各パートには，MML（Music Macro Language）を使って，演奏する音楽のデータを記述します．

基本型は，

```
［#］音名［音長］
```

です（[] 内は省略可能です）．

MMLコマンド

このPLAY3で使えるMMLコマンドは以下のとおりです．

C～B	音名のド～シに対応
#	シャープ（半音上げる）
R	休符
T	テンポの指定
L	デフォルトの音長の指定
O	オクターブの指定
＞	1オクターブ上げる
＜	1オクターブ下げる

●音名について

音名データは以下のように対応しています．

C	D	E	F	G	A	B
ド	レ	ミ	ファ	ソ	ラ	シ

この音名の前に「#」（シャープ）をつけると半音上がります．

●音長について

音長は以下の数値に対応します．

0	32分音符
1	16分音符
2	付点16分音符
3	8分音符
4	付点8分音符
5	4分音符
6	付点4分音符
7	2分音符
8	付点2分音符
9	全音符

音長は音名の後，または，**L**コマンドの後に指定します．

●テンポについて

「**T**」に続いて，「1～255」の数値を指定します．

この値は1が速く，255が遅くなります．

●オクターブについて

「**O**」に続いて，「0～4」の数値を指定します．

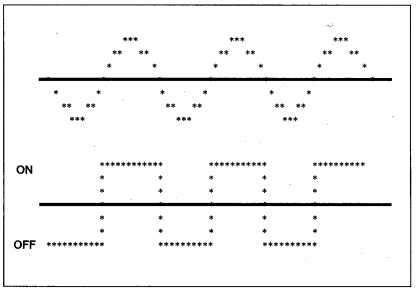
この値は0が最も低く，4が最も高くなります．

音の発生の原理

音は空気が振動して，人間の耳に聞こえていることはご存じですか．

PC-E500の**BEEP**命令では，この音を出すのに本体に内蔵されて圧電ブザーを一定の間隔でON/OFFさせ，振動させることで音を出しています．

たとえば，440Hzの周波数の音（ラ）を出すとすると，およそ2.2msを1サイクル，つまり，1.1msおきにブザーをON/OFFさせることになります．

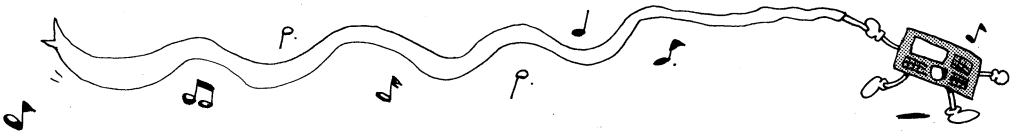


さらに，3音を同時に出すには，3つの音の波形を合成した波形を出せばよいことになります．

もっと簡単にいえば，1音では波形に合わせてブザーをON/OFFさますが，3音のときはそれぞれのパートでON/OFFさせるタイミングのときに実際のブザーの状態を反転させます．

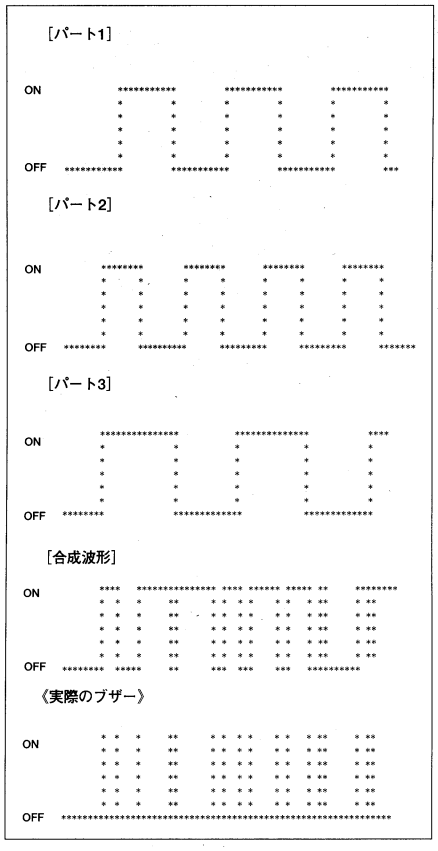
しかし，この方法を実際にプログラムにしてみるとなかなか難しく，処理速度も遅くなってしまいます．

そこで，実際のプログラムでは，ブザーを反転させる代わりに，ブザーをONにしようとするときにブザーをONにして，そのあとすぐにOFFしています．ですから，波形の最初の一瞬だけブザーがONになっています．



いつのポケコンや見当もつかぬPC-1450が￥2．4K，シャープのPB，PC-1246が￥1．4K，そしてPJに投稿の多いPC-G802が￥6．5Kでした．

私はG802を5．8Kにしてもらって買いました．説明書はありませんが，過去のPJの記事，連載をもとに理解しているところです．記憶装置を手に入れたらPJにも投稿していきたいと思います．



PLAY3制作裏話

このPLAY3は，以前，PJ'92年2月号に掲載された「PLAYX」（森田敬太氏作）という3重和音発生のPLAY文を参考に作られています．

というのも，PLAYXを入力して実行してみると，本当にPC-E500本体だけで3和音で音楽が聴こえてくるじゃないですか，これには大変驚きました．

しかし，自分で音楽データ作ってみると，どうも音程が外れていたり，どこもおかしくないのにエラーが出たりして，PLAYXの動作が何だかおかしいことに気付きました．

その後，なんとなくこのPLAYXのバグを取って見ようかとPLAYXを逆アセンブラにかけ解析してみて，バグを取って見たところ，音程の修正にはプログラムのほとんど見直す必要があるということが分かり，諦めてしばらくそのままになっていたところ，松樹さんから「ポケコン本体だけで和音演奏できるプログラムってできないのかなぁ」という話が出ててきて，このことを話したところ「ぜひ，作ってほしい」といわれ，このPLAY3を作ることになりました．

＊

そこで，PLAYXを再度解析し直し，どのようにして3和音を出しているのか調べ，プログラムをすべて書き直していったのです．

この書き直しの途中で，「PLAY2」という2和音発生のPLAY命令も作りました．

これは，最初，3和音ルーチンを作っていたときにプログラムの処理が時間がかかってしまったため，音のサイクルの幅が高音になるほど誤差が大きくなり，音があまり良くなかったので，「2音で音がきれいなほうがいいかな…」と思って作ったものです．

ところが，3音での和音を聞いてしまうと，2音だけでは「何か物足りない」ということで，3和音にしようということになったのです．

しかし，できるだけ2音と同じぐらいの音質にするために，いろいろ試行錯誤を繰り返し，ようやく1サイクルの処理速度がPLAY2とほぼ同じぐらいになりました．この時点でPLAYXのプログラムの原形はほとんどなくなっていました．

これで完成したと思ったのですが，音の発生ルーチンを『活研』に載っているマシン語命令の1命令のサイクル数を見ながら作っていたのに，なぜか高音になればなるほど計算で出した音のサイクル・データでは音が外れるという問題が出てきました．

「まあ，正確にサイクルが合うわけがないから，仕方ないか」とあきらめていましたが，やはり気になって，試しにマシン語命令のサイクル数を自分で数えてみると，なんと『活研』のデータが誤っているということが分かりました．

そこで，音の発生ルーチンを修正したところ，高音もある程度音のズレが直りました．

プログラムについて＆今後の予定

処理を速くするため，プログラムのサイズを少々犠牲にしています．このため，PLAYXに比べ音質がよくなっています．

音程は音の周波数から計算して合わせています．

また，命令のサイクル数を数えているので，今のところ，この音質以上は不可能だと思います．

MMLの仕様は，今回のバージョンでは音長，テンポなどの指定が少々使いづらいかと思います．今後のバージョンアップで，パソコンなどで使われているMMLにできるだけ近いものにしていきたいと思っています．

また，PC-E500以外での機種（PC-E200/G801，PI-ET1，FX-890P/Z-1など）にも，できたら移植してみたいと思っています．このなかでもPC-E200/G801への移植は近いうちに行なう予定です（あくまでも予定なので，あまり期待しないでね）．

おわりに

ソース・リストを見ると気づくと思いますが，公開していないMMLコマンドがいくつかあります．

これは開発中に松樹さんや基さんにテストしてもらい，MMLコマンドをPLAYXと同じにするか，変更するかで調整していたもので，結果的に今回，「曖昧なコマンドは公開しない」ということにしたためです．

「絶対に使ってはいけない」とはいいませんが，今後のバージョンアップ（いつになるのだろうか…）で，MMLデータの互換性がなくなったとき，修正が困難になると思います．

曲データをPJで発表するときは，非公開コマンドは使わないようにしてください．

VX-4とPC-G802を持っている．そんな私は普通科（進学校）高校生です．　（しちどん）

＊

ということで，みなさん，PLAY3でどんどん音楽を作ってPJに発表してくださいね(^^)

■参考資料

1)PC-E500＊1480U活用研究，工学社
2)森田敬太“PLAYX”，PJ'92年2月号

PLAY3のちょっとしたテクニック

●松樹　一純

Ryuさんの力作「PLAY3」を充分に使いこなすために，「知っておくといいかも知れない豆知識」をまとめてみました．

PLAY3のMMLの豆知識

PRINT命令の略式表記を使うときの注意

PLAY命令を拡張すると（PLAY3を組み込むと），それまで**PRINT**命令の略式表記だった「P.」は**PLAY**命令の略式表記になってしまいます．

以後，**EXOFF**で**PLAY**命令を外すまでは，**PRINT**命令の略式表記は「PR.」としなくてはなりません．

プログラムの入力のときには注意しましょう．

リストの要所に見出しをつける

サンプル・ミュージックにあるように，曲データのプログラムの要所に見出し（「＊」に続く文字列）やその旨の表示命令を入れておくと，好きなところからの演奏がしやすいので，プログラムの確認で何かと便利です．

また，これらの見出しを置いた後には，できるだけ，MMLの冒頭にオクターブ指定（**O**）や音長指定（**L**）などの初期設定を置くようにしましょう．

これは，見出しの位置からプログラムを中途実行した場合に，演奏される音の内容が狂わないようにするためです．

MML文の中身の長さは自由

PLAY命令に続くMML文の内容は，必ずしも4分音符4つ分（4／4拍子）である必要はありません．

たとえば，4分音符3つ分（3／4拍子），4分音符6つ分，といったように自由に拍子を想定して書くことができます．

極端な例として，4分音符12個分のような長いMML文や，16分音符1つ分のような短いMML文を書くこともできるわけです．

パートは空白データで省略できる

PLAY3では，同時に3音を発音するために，3パート分のデータを「：」で区切って書くことになっています（詳しくはRyuさんのPLAY3の使い方の項を参照してください）．

たとえば，

```
10:PLAY"C5:E5:G5"
```

という感じですが，2パート，もしくは1パートだけ使って演奏する曲のデータの場合には，他のパートは表記を省略することができるようになっています．

上記の例から「**E**」の演奏パートを省く場合，

```
10:PLAY"C5:G5"
```

のようになります．

また，次のような書き方もできます．

```
10:PLAY"C5::G5"
```

もしくは，

```
10:PLAY"C5:G5:"
```

このような表記方法は，「あとからもう1パート付け加えたいが，とりあえず，先に他のパートを入力・演奏させよう」といった場合に便利です．

ちなみに，

```
10:PLAY":C5:G5"
```

のように，MMLの冒頭に「：」が来ると，正しく演奏されないので注意しましょう．

パートごとの音長の合計が異なる場合

ひとつの**PLAY**命令（MML文）の中で，各パートごとの音長の合計が異なる場合は，いちばん長いパートが終了するまで，他のパートはその長さの差分を休符と同様に無音状態で，次の**PLAY**文の実行を待ちます．

たとえば，

```
10:PLAY"C5:E3:G3"
20:PLAY"C5:E5:G5"
```

という処理があったとすると，10行目の演奏の「**E**」や「**G**」は「**C**」の半分の長さしかなく，8分音符1つ分のデータの空白がありますが，こういう場合，

```
10:PLAY"C5:E3R3:G3R3"
20:PLAY"C5:E5:G5"
```

と書いたときの演奏と同じになります．

オクターブ違いの同じ音を同時に出すときの注意

オクターブ違いの同じ音同士を同時に出すと，“食い合せ”が悪いらしく，お互いの音が溶け込んで，他の異なる音同士を同時に出したときと比べて音が薄く（小さく）なってしまいます．

曲を通して聴いたときに，このような場所だけ音が薄く聴こえてしまうので，作曲のときには，異なる音をかぶせるなどしてフォローするとGoodです．

曲の流れにめりはりをつけるには

ビープ音は単調なので，ただ音を演奏させるだけではどうしても曲にめりはりがつきづらいものがあります．

そこで，アクセントをつける簡単な例として，たとえば，次のような手法があります．

PJプラザ

▼「ポケ・ジャー’93年10月号を見ました」
p.48に私の記事が載って嬉しかった…．
その次のp.49の岐阜県のモアイさん，『G815で画面が左に寄る」と書いてありましたが，ポケ・ジャー’93年4月号の小林龍也氏が「フルドットマトリックスになり，LCDコントローラが変更され，連続してキャラクタを書くと左に寄る」と書いておられました．

```
10:PLAY"L5CGEG:L5CCCC"
```

というような演奏があったとき，一音一音に歯切れを持たせるため，最初のパート（メロディ）の音にそれぞれ休符を噛ませて，

```
10:PLAY"L3CRGRERGR:L5CCCC"
```

という風に「1つ1つの音を意識しやすくする」――という方法です．

なんのことはない簡単な工夫ですが，ときにはなかなか効果的です．

疑似和音でもう1パート分音を多く出す

ちょっと苦しい方法ですが，テンポの値によっては，短い音を交互に並べることで，それらの音の和音が鳴っているように聴かせることができます．

たとえば，「オクターブ2のCとEの和音，音長は4分音符分の長さ」，

```
10:PLAY"L502C:L5O2E"
```

という2パート分の演奏をなんとか1パートでこなしたいときには，

```
10:PLAY"L0O2CECECECE:L5R"
```

と置き換えることで，それに近い耳ざわりの音を得ることができるわけです．

ただし，これは短い音の差し替えで出した“ごまかし”でしかないので，普通に和音を演奏したときに比べて，音がブレだような感じになります．

これを逆に効果として，ビブラート風のブレが音に欲しいときに利用してみてもいいでしょう．

＊

PLAY3はアイデア次第で面白い表現ができる演奏ルーチンです．

また，BASICの拡張命令になっているので気軽に利用できます．

ぜひ，このソフトを活用して，あなたもポケコン・ミュージックにチャレンジしてみてください．

PLAY3の演奏を録音するには

通常のBASICプログラムをテープに記録する要領で，本体とカセット・インターフェイス，テープ・レコーダーを接続すれば，PLAY3から流れ出る演奏をテープに録音することができます．

PLAY3の演奏を大きな音で聴くには（裏技）

●この楽しみ方はあくまで裏技なので，実行にあたっては，各自の判断のもとに行なってください．

万一，この方法によって何らかの損失などがあっても，当方は責任を負いかねます．

まず，本体にE500用のカセット・インターフェイス（CE-124など）を接続し，ピンジャック（マイク側…赤色）をラジカセやアンプなどのLINE-IN端子につなぎます．

あとはそれらのボリュームを最小にした状態でE500側でPLAY3の演奏を開始し，徐々に音を上げて，聴きやすい音量に調節すればOKです．

＊

このとき，最初からアンプ類のボリュームを大きくしてPLAY3の演奏を流すと，接続したオーディオ機器を傷める恐れがあるので要注意！

また，接続のときには電源のON/OFFにも気をつけましょう．

MIDI関係の機材を持っている人は，各種エフェクターを接続するなどして，ひと味違ったポケコン演奏を楽しんでみてください．

特にリバーブ（残響効果）をかけての演奏は音に幅が出るので，オススメです．

音楽プログラム募集のお知らせ

かねてより誌上でお知らせしたとおり，近く，ポケコン・ミュージックのシリーズ連載を開始する予定です．

これにあたって，読者の皆さんからのミュージック・プログラムを募集します．

＊

今回発表されたRyuさんの「PLAY3」をはじめ，これまでにPJ誌上で発表された音楽演奏ルーチンを使ったミュージック・プログラムの形で投稿してください．

また，これからの新しい標準になるような，音楽演奏ツールの力作も随時募集します．

＊

ミュージック・プログラムの内容は，できればオリジナルの曲が望ましいのですが，場合によっては既存のゲーム・ミュージックなどの投稿もOKです．ただし，この場合は，必ず原曲の出典を明記してください．

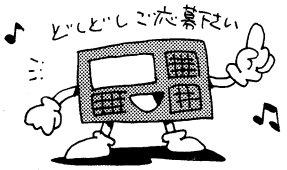
また，オリジナル曲の場合，「この曲はみんなのポケコン・ゲームの中で自由に使ってもらいたい」というような曲の提供も大歓迎です．投稿のときに明記してください．

＊

ミュージック・コーナーは当面，皆さんからのミュージック・プログラムの投稿を待って，「ゲーム・デザイン館」のような不定期連載の形を取っていく予定です．

ミュージック・コーナーを盛り上げる楽しいアイデアも大歓迎です．

皆さんからの応募,お待ちしています．



小林様！　あの記事とっても役に立っております．感謝しています！
PS　そろそろプログラム送ります．

小林氏の記事を読んでいた（ニノキン）

リスト1　PLAY3 BASICリスト

```
100  CLS:PRINT "******** PLAY3 ****
     ****"
110  L=1000:S=&HBF000
120  '
130  LOCATE 0, 2:PRINT "Line No.";L
140  SUM=0
150  FOR J=0 TO 7
160  READ D$:POKE (S+J), VAL ("&H"+
     D$)
170  IF D$="END" THEN PRINT "Comple
     ted":END
180  'LOCATE 0, 3:PRINT "Writing Add
     ress =";S+J
190  SUM=SUM+VAL ("&H"+D$)
200  NEXT J
210  READ D$:IF SUM<> VAL ("&H"+D$)
      THEN PRINT "Error":BEEP 1:END
220  L=L+10:S=S+8
230  GOTO 130
1000 DATA 0C, 26, F0, 0B, 30, BC, 80, D1, 3
     6A ' BF000
1010 DATA 90, 0C, 35, F0, 0B, 30, BC, 80, 3
     38 ' (A2)
1020 DATA D1, 93, 07, 0C, FF, FF, 0F, 30, 3
     B4 ' BF010
1030 DATA BC, 80, D1, 90, 30, BC, 80, D1, 4
     DA ' (8E)
1040 DATA 93, 05, FA, F1, 0B, 07, 04, 50, 2
     E9 ' BF020
1050 DATA 4C, 41, 59, 1F, 05, 45, 58, 4F, 1
     F6 ' (DF)
1060 DATA 46, 46, 02, 00, 00, 1F, 3F, F0, 1
     DC ' BF030
1070 DATA 8B, 02, 13, F0, 8B, 00, 00, 2F, 2
     4A ' (26)
1080 DATA 30, CC, FB, A0, 30, E8, 36, EC, 4
     D1 ' BF040
1090 DATA 04, 57, F0, 2C, 05, 19, F2, 0B, 2
     92 ' (63)
1100 DATA 3C, 30, E0, 26, EC, 3F, 07, 0D, 2
     B1 ' BF050
1110 DATA EE, F1, 0B, 30, A5, 30, 0D, EF, 3
     EB ' (9C)
1120 DATA F1, 0B, 30, A5, 33, 0D, 19, F5, 3
     1F ' BF060
1130 DATA 0B, AD, F4, F1, 0B, 0D, ED, F1, 4
     93 ' (B2)
1140 DATA 0B, AD, 39, F5, 0B, AD, 29, F5, 3
     BC ' BF070
1150 DATA 0B, 0D, 47, F5, 0B, AD, 19, F5, 3
     1A ' (D6)
1160 DATA 0B, 30, CC, 13, FF, 90, 24, 60, 3
     2D ' BF080
1170 DATA 0D, 18, 99, 60, 22, 18, 02, 7C, 1
     D6 ' (03)
1180 DATA 04, 90, 24, 60, 22, 18, 8F, 60, 2
     41 ' BF090
1190 DATA 0D, 18, 89, 60, 3A, 18, B2, 60, 2
     72 ' (B3)
1200 DATA 3E, 14, CA, F1, 60, 3C, 14, D7, 3
     94 ' BF0A0
1210 DATA F1, 60, 4F, 14, B1, F1, 60, 54, 4
     0A ' (9E)
1220 DATA 18, DA, 60, 4C, 18, 75, 60, 52, 2
     DD ' BF0B0
1230 DATA 18, 66, 60, 2B, 14, C5, F1, 60, 3
     33 ' (10)
1240 DATA 2D, 14, D2, F1, 60, 49, 18, 7B, 3
     40 ' BF0C0
1250 DATA 2D, 0D, DF, F1, 0B, 60, 23, 1A, 2
     B2 ' (F2)
1260 DATA 06, 0D, E6, F1, 0B, 90, 24, 48, 2
     F1 ' BF0D0
1270 DATA 41, 60, 07, 1E, 40, 45, 50, 90, 2
     2B ' (1C)
1280 DATA 05, FD, 10, 3D, 90, 24, 48, 30, 2
     7B ' BF0E0
1290 DATA 60, 0A, 1C, 10, 48, 25, 60, 06, 1
     69 ' (E4)
1300 DATA 1E, 04, 40, 0A, 12, 06, 30, 98, 1
     4C ' BF0F0
1310 DATA 00, 33, 7C, 04, 28, 88, F7, F1, 3
     4B ' (97)
1320 DATA 0B, 60, 00, 38, 1A, 04, 30, B8, 1
     A9 ' BF100
1330 DATA 00, 33, 46, 01, B0, 25, 30, 61, 1
     E0 ' (89)
1340 DATA 13, 10, 1F, 83, 30, 80, 13, 30, 1
     B8 ' BF110
1350 DATA CC, 13, FF, 12, 98, 3D, 13, 8F, 3
     67 ' (1F)
1360 DATA 09, 00, 13, 40, 7C, 04, 08, FF, 1
     E3 ' BF120
1370 DATA B0, 25, 06, 90, 04, 48, 30, 60, 2
     47 ' (2A)
1380 DATA 0A, 1C, 08, 48, 55, 60, 06, 1F, 1
     50 ' BF130
1390 DATA A8, 40, 0A, 30, B8, 00, 33, 6C, 2
     79 ' (C9)
1400 DATA 04, 13, B2, 90, 04, 48, 30, 1C, 1
     F1 ' BF140
1410 DATA 04, A8, F7, F1, 0B, 6C, 04, 13, 3
     22 ' (13)
1420 DATA C0, 2C, 08, FF, B0, 25, 8C, F4, 4
     48 ' BF150
1430 DATA F1, 0B, 0A, 10, 00, 45, 42, AC, 2
     49 ' (91)
1440 DATA F4, F1, 0B, B5, 04, 30, 84, 30, 3
     8D ' BF160
1450 DATA 6C, 04, 6C, 04, 30, A4, 30, 30, 2
     14 ' (A1)
1460 DATA 84, 33, 6C, 04, 6C, 04, 30, A4, 2
     6B ' BF170
1470 DATA 33, 3C, 13, EB, 90, 04, 48, 30, 2
     79 ' (E4)
1480 DATA 60, 0A, 1F, F3, 40, D0, B0, 25, 3
     61 ' BF180
1490 DATA 6C, 04, 13, FB, 09, 00, 08, DF, 2
     6E ' (CF)
1500 DATA B0, 25, 90, 04, 48, 30, 60, 0A, 2
     4B ' BF190
1510 DATA 1E, 12, 28, FD, 01, 46, 00, 46, 1
     E2 ' (2D)
1520 DATA 00, 46, 10, 46, 11, 38, 46, 10, 1
     3B ' BF1A0
1530 DATA 6C, 04, 13, 1A, B1, 25, 02, 91, 2
     06 ' (41)
1540 DATA F0, 90, 24, 48, 30, 60, 0A, 17, 2
     9D ' BF1B0
1550 DATA 91, F0, 30, B8, 00, 30, 40, E0, 3
     B9 ' (56)
1560 DATA B0, 25, 02, 91, F0, 32, F0, 00, 3
     7A ' BF1C0
1570 DATA 13, 30, 30, 98, 00, 30, 6C, 00, 1
     A7 ' (21)
1580 DATA 13, 1D, 32, F0, 00, 13, 30, 30, 1
     C5 ' BF1D0
1590 DATA 98, 00, 30, 7C, 00, 13, 2A, A0, 2
     21 ' (E6)
1600 DATA C0, 10, 30, 50, 60, 80, B0, 10, 2
     F0 ' BF1E0
1610 DATA 20, 40, 50, 70, 90, FF, 03, 05, 2
     B7 ' (A7)
1620 DATA 03, 05, 03, 05, 00, 00, 00, 01, 0
     11 ' BF1F0
1630 DATA 1F, 1F, 0D, 00, 00, 00, AD, 1F, 1
     0A ' (1B)
1640 DATA F5, 0B, AD, 2F, F5, 0B, AD, 3F, 3
     C8 ' BF200
1650 DATA F5, 0B, 08, 02, A8, 1C, F5, 0B, 2
     CE ' (96)
1660 DATA A8, 2C, F5, 0B, A8, 3C, F5, 0B, 3
     B8 ' BF210
1670 DATA 07, 0B, 00, 00, AB, 1D, F5, 0B, 1
     DA ' (92)
1680 DATA AB, 2D, F5, 0B, AB, 3D, F5, 0B, 3
     C0 ' BF220
1690 DATA 8D, 19, F5, 0B, AD, 16, F5, 0B, 3
     69 ' (29)
1700 DATA 8D, 29, F5, 0B, AD, 26, F5, 0B, 3
     89 ' BF230
1710 DATA 8D, 39, F5, 0B, AD, 36, F5, 0B, 3
     A9 ' (32)
1720 DATA 0C, 16, F5, 0B, 04, 66, F3, 0C, 2
     8B ' BF240
1730 DATA 26, F5, 0B, 04, 66, F3, 0C, 36, 2
     C5 ' (50)
1740 DATA F5, 0B, 04, 66, F3, 08, 00, 75, 2
     DA ' BF250
1750 DATA 88, 1E, F5, 0B, 30, D0, 00, 2E, 2
     D4 ' (AE)
1760 DATA F5, 0B, 30, 61, 00, 00, 18, 08, 1
     B1 ' BF260
1770 DATA 30, 63, 00, 1E, 03, 30, 80, 00, 1
     64 ' (15)
1780 DATA 30, D0, 00, 3E, F5, 0B, 30, 61, 2
     CF ' BF270
1790 DATA 00, 00, 18, 08, 30, 63, 00, 1E, 0
     D1 ' (A0)
1800 DATA 03, 30, 80, 00, 60, 00, 18, DD, 2
     08 ' BF280
1810 DATA FD, 10, 88, 1E, F5, 0B, 4E, 01, 3
     02 ' (0A)
1820 DATA A8, 1E, F5, 0B, 88, 2E, F5, 0B, 3
     7C ' BF290
1830 DATA 4E, 01, A8, 2E, F5, 0B, 88, 3E, 2
     EB ' (67)
1840 DATA F5, 0B, 4E, 01, A8, 3E, F5, 0B, 3
     35 ' BF2A0
1850 DATA 44, 33, 88, 0B, F5, 0B, 0C, 00, 2
     16 ' (4B)
1860 DATA 00, 00, 44, 33, 9F, F4, 1E, 02, 2
     2A ' BF2B0
1870 DATA 45, 43, 60, 00, 1B, 0C, AC, 0C, 1
     C7 ' (F1)
1880 DATA F5, 0B, 30, CC, 00, 00, 0D, 0F, 2
     18 ' BF2C0
1890 DATA F5, 0B, 88, 1D, F5, 0B, 6C, 00, 3
     11 ' (29)
1900 DATA 18, 0D, 7C, 00, 18, 09, 8A, 1F, 1
     6B ' BF2D0
1910 DATA F5, 0B, B2, 25, 30, 6D, 00, 88, 2
     FC ' (67)
1920 DATA 2D, F5, 0B, 6C, 00, 18, 0D, 7C, 2
     3A ' BF2E0
1930 DATA 00, 18, 09, 8A, 2F, F5, 0B, B2, 2
     8C ' (C6)
1940 DATA 25, 30, 6D, 00, 88, 3D, F5, 0B, 2
     87 ' BF2F0
1950 DATA 6C, 00, 18, 0D, 7C, 00, 18, 09, 1
     2E ' (B5)
1960 DATA 8A, 3F, F5, 0B, B2, 25, 30, 6D, 3
     3D ' BF300
1970 DATA 00, 8C, 0C, F5, 0B, 8A, 0F, F5, 3
     26 ' (63)
1980 DATA 0B, FD, 32, FD, 53, AA, F4, F3, 5
     1B ' BF310
1990 DATA 0B, AB, 3F, F4, 0B, AB, FF, F3, 4
     91 ' (AC)
2000 DATA 0B, AD, 06, F4, 0B, AD, 27, F4, 3
     85 ' BF320
2010 DATA 0B, AD, 46, F4, 0B, AD, 5A, F4, 3
     F8 ' (7D)
2020 DATA 0B, 30, 7D, 00, 14, EF, F3, 8B, 3
     39 ' BF330
2030 DATA 11, F5, 0B, AB, 3F, F4, 0B, AB, 3
     A5 ' (DE)
2040 DATA FF, F3, 0B, 7C, 03, 7C, 03, 7C, 3
     77 ' BF340
2050 DATA 03, 30, 7D, 00, 14, EF, F3, 8A, 3
     30 ' (A7)
2060 DATA 13, F5, 0B, AA, F4, F3, 0B, 7C, 4
     2B ' BF350
2070 DATA 02, 7C, 02, 7C, 02, 30, 7D, 00, 1
     AB ' (D6)
2080 DATA 14, EF, F3, 12, 75, 07, 2D, 90, 3
     41 ' BF360
2090 DATA 84, 07, 60, FF, 18, 26, 90, 84, 3
     3C ' (7D)
2100 DATA 08, 60, 00, 1A, 2A, 95, 04, 90, 1
     D5 ' BF370
2110 DATA 25, 60, FF, 18, 17, 60, DF, 18, 3
     0A ' (DF)
2120 DATA 0B, 60, E0, 1C, 20, 70, 0F, B0, 2
     B6 ' BF380
2130 DATA 84, 06, 13, 15, 90, 25, A8, 0B, 2
     1A ' (D0)
2140 DATA F5, 0B, 13, 1D, B0, 84, 07, 08, 2
     73 ' BF390
2150 DATA 00, B0, 84, 08, 3D, 97, 06, 93, 2
     A9 ' (1C)
2160 DATA 84, 09, 3D, 9F, 06, B5, 04, 28, 2
     50 ' BF3A0
2170 DATA 70, F0, E4, E4, FD, 10, E4, E4, 5
     FD ' (4D)
2180 DATA B0, 84, 07, 46, 01, 46, 00, 0D, 1
     D5 ' BF3B0
2190 DATA 79, F4, 0B, 45, 50, 90, 84, 06, 3
     27 ' (FC)
2200 DATA 46, 00, 45, 50, 93, 05, B3, 84, 2
     AA ' BF3C0
2210 DATA 09, 38, 70, 0F, 0D, FB, F4, 0B, 2
     C7 ' (71)
2220 DATA 45, 50, 90, 05, B0, 84, 08, 3D, 2
     A3 ' BF3D0
2230 DATA 9F, 06, 0B, 04, 00, 7C, 03, 1B, 1
     4E ' (F1)
2240 DATA 04, 30, CC, FD, 02, 00, 00, 00, 1
     FF ' BF3E0
2250 DATA 7C, 04, 1B, 12, 02, 40, F2, 7C, 2
     5D ' (5C)
2260 DATA 02, 1A, 41, 0A, 00, 00, 30, CC, 1
     63 ' BF3F0
2270 DATA FD, 02, 7C, 03, 1A, 22, 0B, 00, 1
     C5 ' (28)
2280 DATA 00, 7C, 05, 1A, 0E, 0D, 00, 00, 0
     B6 ' BF400
2290 DATA 00, 30, CC, FD, 12, 7C, 04, 1B, 2
     A6 ' (5C)
2300 DATA 22, 12, 5F, 00, 00, 00, 30, CC, 1
     8F ' BF410
2310 DATA FD, 12, 7C, 04, 1B, 2F, 12, 52, 2
     3D ' (CC)
2320 DATA 00, 00, 7C, 05, 1B, 13, 0D, 00, 0
     BC ' BF420
2330 DATA 00, 00, 30, CC, FD, 12, 7C, 04, 2
     8B ' (47)
2340 DATA 1B, 43, 12, 3E, 00, 00, 30, CC, 1
     AA ' BF430
2350 DATA FD, 02, 7C, 03, 1A, 15, 0B, 00, 1
     B8 ' (62)
2360 DATA 00, 7C, 05, 1B, 32, 0D, 00, 00, 0
     DB ' BF440
2370 DATA 00, 30, CC, FD, 12, 7C, 04, 1B, 2
     A6 ' (81)
2380 DATA 62, 12, 1F, 00, 00, 7C, 05, 1A, 1
     2E ' BF450
2390 DATA 0E, 0D, 00, 00, 00, 30, CC, FD, 2
     14 ' (42)
2400 DATA 12, 7C, 04, 1B, 76, 12, 0B, 00, 1
     40 ' BF460
2410 DATA 00, 00, 00, 00, 00, 00, 7C, 04, 0
     80 ' (C0)
2420 DATA 1B, 83, 30, CC, FD, 02, 02, 40, 2
     DB ' BF470
2430 DATA F2, 00, 00, 00, 00, 00, 00, 00, 0
     F2 ' (CD)
2440 DATA 00, 00, 00, 2E, 01, 97, 00, 4C, 1
     12 ' BF480
2450 DATA 00, 26, 00, 13, 00, 1D, 01, 8F, 0
     E6 ' (F8)
2460 DATA 00, 48, 00, 24, 00, 12, 00, 0D, 0
     8B ' BF490
2470 DATA 01, 87, 00, 44, 00, 22, 00, 11, 0
     FF ' (8A)
2480 DATA 00, FE, 00, 7F, 00, 40, 00, 20, 1
     DD ' BF4A0
2490 DATA 00, 10, 00, F0, 00, 78, 00, 3C, 1
     B4 ' (91)
2500 DATA 00, 1E, 00, 0F, 00, E2, 00, 71, 1
     80 ' BF4B0
2510 DATA 00, 39, 00, 1D, 00, 0F, 00, D6, 1
     3B ' (BB)
2520 DATA 00, 6B, 00, 36, 00, 1B, 00, 0E, 0
     CA ' BF4C0
2530 DATA 00, CA, 00, 65, 00, 33, 00, 1A, 1
     7C ' (46)
2540 DATA 00, 0D, 00, BE, 00, 5F, 00, 30, 1
     5A ' BF4D0
2550 DATA 00, 18, 00, 0C, 00, B4, 00, 5A, 1
     32 ' (8C)
2560 DATA 00, 2D, 00, 17, 00, 0C, 00, AA, 0
     FA ' BF4E0
2570 DATA 00, 55, 00, 2B, 00, 16, 00, 0B, 0
     A1 ' (9B)
2580 DATA 00, A0, 00, 50, 00, 28, 00, 14, 1
     2C ' BF4F0
2590 DATA 00, 0A, 00, 03, 06, 09, 0C, 12, 0
     3A ' (66)
2600 DATA 18, 24, 30, 48, 60, 02, 04, 06, 1
     20 ' BF500
2610 DATA 10, 20, 01, 01, 00, 00, 00, 00, 0
     32 ' (52)
2620 DATA 00, 00, 00, 00, 00, 00, 00, 00, 0
     00 ' BF510
2630 DATA 00, 00, 00, 00, 00, 00, 00, 00, 0
     00 ' (00)
2640 DATA 00, 00, 00, 00, 00, 00, 00, 00, 0
     00 ' BF520
2650 DATA 00, 00, 00, 00, 00, 00, 00, 00, 0
     00 ' (00)
2660 DATA 00, 00, 00, 00, 00, 00, 00, 00, 0
     00 ' BF530
2670 DATA 00, 00, 00, 00, 00, 00, 00, 00, 0
     00 ' (00)
2680 DATA 00, 00, 00, 00, 00, 00, 00, 00, 0
     00 ' BF540
2690 DATA 00, 00, 00, 00, 00, 00, 00, 00, 0
     00 ' (00)
2700 DATA END
```

PJプラザ

▼初投稿です．

所有機種はFX-890P，G801です（FX870Pは溺死）．

さっそく，'93年10月号p．35のINGRAMさん，マシン語ファイルとソースファイルとの区別は16C6hから10バイト使って，マシン語ファイルのところだけ0Dhが入っています．

リスト2　PLAY3 ソース・リスト（XASM用）

```
  1:                pre_on                  ; プレバイト自動生成機能ＯＮ
  2: ;-----------------------------------------------------------------
  3: ;----- Systemエントリー -----
  4: fcs:           equ  0fffe4h            ; File Controll System エントリー
  5: iocs:          equ  0fffe8h            ; Input Output Controll System エントリー
  6: ;----- 割り込みベクタ -----
  7: vect0:         equ  0bfcc6h            ; fast タイマ割り込み
  8: vect1:         equ  0bfcc9h            ; slow タイマ割り込み
  9: vect2:         equ  0bfccch            ; キー割り込み
 10: vect3:         equ  0bfccfh            ; ＯＮキー割り込み
 11: vect4:         equ  0bfcd2h            ; ＳＩＯ送信割り込み
 12: vect5:         equ  0bfcd5h            ; ＳＩＯ受信割り込み
 13: vect6:         equ  0bfcd8h            ; 外部割り込み
 14: vect7:         equ  0bfcdbh            ; ソフトウェア割り込み
 15: ;----- 内部ＲＡＭレジスタ -----
 16: bx:            equ  0d4h
 17: bh:            equ  0d5h
 18: cx:            equ  0d6h
 19: ch:            equ  0d7h
 20: dx:            equ  0d8h
 21: dh:            equ  0d9h
 22: si:            equ  0dah
 23: di:            equ  0ddh
 24: ;----- ポインタ -----
 25: abp:           equ  0ech
 26: apx:           equ  0edh
 27: apy:           equ  0eeh
 28: ;----- 割り込みレジスタ -----
 29: iisr:          equ  0ebh               ; 割り込みサービス中・レジスタ
 30: imr:           equ  0fbh               ; 割り込みイネーブル・レジスタ
 31: isr:           equ  0fch               ; 割り込みステータス・レジスタ
 32: ;-----  -----
 33: beep:          equ  0fdh
 34: basic_work:    equ  0d1h
 35: cr:            equ  00dh
 36: ;-----  -----
 37: part_c:        equ  0                  ; パートカウンタ
 38: data_adr:      equ  0                  ; ＭＭＬデータアドレス(3byte)
 39: data_s_adr:    equ  data_adr+3         ; ＭＭＬデータ先頭アドレス(3byte)
 40: oct:           equ  data_s_adr+3       ; オクターブ（０）(1byte)
 41: onp:           equ  oct+1              ; 音符コード(1byte)
 42: l_count:       equ  onp+1              ; 音長カウンタ(1byte)
 43: o_count:       equ  l_count+1          ; 音程カウンタ(3byte)
 44: ;-----------------------------------------------------------------
 45: ;     ＭＭＬ　内部ＲＡＭ　ＷＯＲＫ
 46: ;-----------------------------------------------------------------
 47: oct_flg:       equ  13h                ; オクターブ設定フラグ
 48: octerve_adr:   equ  30h                ; オクターブアドレス
 49: length_adr:    equ  33h                ; レングスアドレス
 50: ;
 51: e_flg:         equ  18h                ; 終了フラグ
 52: ;-----------------------------------------------------------------
 53:                org  0bf000h
 54: ;-----------------------------------
 55: ;     インストール
 56: ;-----------------------------------
 57: install:       local                   ; ＢＡＳＩＣコマンド組込み(PLAY,EXOFF)
 58:                mv   x,basic_command
 59:                mv   [(!basic_work)+90h],x
 60:                mv   x,basic_code
 61:                mv   [(!basic_work)+93h],x
 62:                retf
 63: remove:        mv   x,0ffffh           ; ＢＡＳＩＣコマンド解除
 64:                mv   [(!basic_work)+90h],x
 65:                mv   [(!basic_work)+93h],x
 66:                calf !init1
 67:                retf
 68: ;
 69: ;-----------------------------------
 70: ;     ＢＡＳＩＣコマンドテーブル
 71: ;-----------------------------------
 72: basic_command: db                      04h ; 命令の文字数
 73:                dm   'PLAY'             ; 命令の文字列
 74:                db   1fh                ; 中間コード
 75:                db   5                  ; 命令の文字数
 76:                dm   'EXOFF'            ; 命令の文字列
 77:                db   2                  ; 中間コード
 78:                db   0,0                ; コマンドテーブル終了コード
 79: basic_code:    db   1fh                ; 中間コード
 80:                dw   !entry             ; 処理先アドレス
 81:                db   8bh                ; bit6=1 関数 bit7=1 コマンド処理
 82:                db   2                  ; 中間コード
 83:                dw   remove             ; 処理先アドレス
 84:                db   8bh                ; bit6=1 関数 bit7=1 コマンド処理
 85:                db   0,0                ; 処理先テーブル終了コード
 86:                endl
 87: ;-----------------------------------
 88: ;     初期設定
 89: ;-----------------------------------
 90: entry:         local
 91:                pushuimr                ; 割り込みイネーブル・レジスタ保存
 92:                mv   (!imr),0a0h        ; 割り込み禁止
 93:                mv   [--u],(!abp)       ; bp 保存
 94:                call mml_conv           ; MML解析、処理
 95: ;
 96:                pushu                   x
 97:                calf !init2             ; 初期化
 98:                popu                    x
 99:                mv   (!abp),[u++]       ; bp 復帰
100:                popuimr                 ; 割り込みイネーブル・レジスタ復帰
101:                                        ; 割り込み許可
102:                retf
103: ;-----------------------------------------------------------------
104: ;     ＭＭＬ　Ｃｏｎｖｅｒｔｅｒ
105: ;-----------------------------------------------------------------
106: mml_conv:     mv   y,octerve
107:                mv   (!octerve_adr),y   ; オクターブデータアドレス設定
108:                mv   y,length
109:                mv   (!length_adr),y    ; レングスデータアドレス設定
110: ;
111:                mv   y,!part1+!data_s_adr
112:                mv   [MML_data_adr],y   ; MML解析データ設定アドレス
113:                mv   y,not_data
114:                mv   [!part3+!data_s_adr],y ; 3 音目データアドレス
115:                mv   [!part2+!data_s_adr],y ; 2 音目データアドレス
116:                mv   y,!data_buff
117:                mv   [!part1+!data_s_adr],y ; 1 音目データアドレス
118: ;-----------------------------------
119: ;     ＭＭＬ解析
120: ;-----------------------------------
121:                mv   (!oct_flg),0ffh
122:                mv   a,[x++]            ; MMLデータ読み込み
123:                cmp a,!cr
124:                jrz  mml_conv_exit_c    ; パラメータが無い
125:                cmp a,'"'
126:                jrz  mml_conv_lp        ; ダブルクォーテーションをスキップ
127:                dec  x
128: mml_conv_lp:mv  a,[x++]
129:                cmp a,'"'
130:                jrz  mml_conv_exit      ; ダブルクォーテーションがあったら終了
131:                cmp a,!cr
132:                jrz  mml_conv_exit_c    ; ｃｒがあったら終了
133:                cmp a,':'
134:                jrz  part_ch            ; part チェンジ
135:                cmp a,'>'
136:                jpz  oct_up             ; １オクターブＵＰ
137:                cmp a,'<'
138:                jpz  oct_dw             ; １オクターブＤＯＷＮ
139:                cmp a,'O'
140:                jpz  oct_ch             ; オクターブ変更
141:                cmp a,'T'
142:                jrz  temp2_ch           ; テンポ変更
143:                cmp a,'L'
144:                jrz  len_ch             ; デフォルトレングス変更
145:                cmp a,'R'
146:                jrz  kyuufu             ; 休符
147:                cmp a,'+'
148:                jpz  oct_1up
149:                cmp a,'-'
150:                jpz  oct_1dw
151:                cmp  a,'I'
152:                jrz  format_ch
153: ;-----------------------------------
154: ;     音符データ設定
155: ;-----------------------------------
156: onp_set:      pushu y
157:                mv   y,mml_data1        ; 音程データ先頭アドレス
158:                cmp  a,'#'
159:                jrnz onp_skip           ; シャープ？
160:                mv   y,mml_data2        ; シャープ音程データ先頭アドレス
161:                mv   a,[x++]
162: onp_skip:     sub  a,'A'
163:                cmp  a,7
164:                jrnc onp_skip2
165:                add  y,a
166:                mv   a,[y]
167:                mv   il,a               ; 音程に対応した音程データ番号を計算
168:                popu y
169: onp_len:      mv   a,[x++]
170:                sub  a,'0'
171:                cmp  a,10
172:                jrc  onp_len_sk2        ; 音符の後に音長の指定がある
173:                sub  a,'U'-'0'
174:                cmp  a,6
175:                jrnc onp_len_sk
176:                add  a,10
177:                jr   onp_len_sk2
178: onp_len_sk:   mv   a,[(!length_adr)]
179:                dec  x                  ; デフォルトの音長
180: onp_len_sk2:  pushu a                 ; 音長取得
181:                mv   a,[format_data]
182:                cmp  a,0
183:                popu a
184:                jrnz onp_len_sk3
185:                mv   [(!length_adr)],a  ; length デフォルト設定
186: onp_len_sk3:  add  a,il
187:                mv   [y++],a            ; データセット
188:                cmp  (!oct_flg),010h
189:                jrnc mml_conv_lp
190:                mv   a,(!oct_flg)
191:                mv   (!oct_flg),0ffh
192:                jr   oct_ch_s
193: onp_skip2:    popu y
194:                jr   mml_conv_lp
195: ;-----------------------------------
196: ;     休符設定
197: ;-----------------------------------
198: kyuufu:       mv   il,0
199:                jr   onp_len
200: ;-----------------------------------
201: ;     終了処理
202: ;-----------------------------------
203: mml_conv_exit_c:dec  x
204: mml_conv_exit: mv  a,0ffh
205:                mv   [y++],a
206:                ret                     ; ＭＭＬデータ解析終了
207: ;-----------------------------------
208: ;     音長設定
209: ;-----------------------------------
210: len_ch:       mv   a,[x]
211:                sub  a,'0'
212:                cmp  a,0ah
213:                jrc  len_sk
214:                sub  a,'U'
215:                cmp  a,6
216:                jrnc mml_conv_lp
217:                add  a,10
218: len_sk:       mv   [(!length_adr)],a
219:                inc  x
220:                jr   mml_conv_lp
221: ;-----------------------------------
222: format_ch:    mv   a,[x]
223:                sub  a,'0'
224:                jrc  format_sk
225:                mv   [format_data],a
226: format_sk:    inc  x
227:                jr   mml_conv_lp
228: ;-----------------------------------
229: ;     パート変更処理
230: ;-----------------------------------
231: part_ch:      pushu x
232:                mv   a,0ffh
233:                mv   [y++],a
234:                mv   x,[MML_data_adr]
235:                mv   ba,16
236:                add  x,ba
237:                mv   [MML_data_adr],x
238:                mv   [x],y
239:                mv   x,(!octerve_adr)
240:                inc  x
241:                inc  x
242:                mv   (!octerve_adr),x
243:                mv   x,(!length_adr)
244:                inc  x
245:                inc  x
246:                mv   (!length_adr),x
247:                popu x
248:                jr   mml_conv_lp
249: ;-----------------------------------
250: ;     テンポ設定
251: ;-----------------------------------
252: temp_ch:      mv   a,[x]
253:                sub  a,30h
254:                cmp  a,0ah
255:                jrnc mml_conv_lp
256:                add  a,0d0h
257:                mv   [y++],a
258:                inc  x
259:                jr   mml_conv_lp
260: ;-----------------------------------
261: temp2_ch:     mv   il,0
262:                mv   a,0dfh
263:                mv   [y++],a
264: temp2_lp1:    mv   a,[x]
265:                sub  a,30h
266:                cmp  a,10
267:                jrnc temp2_sk1
268:                pushu a
269:                mv   a,il
270:                add  a,a
271:                add  a,a
272:                add  il,a
273:                add  il,il
274:                popu a
275:                add  il,a
276:                inc  x
277:                jr   temp2_lp1
278: temp2_sk1:    mv   [y++],il
279:                jp   mml_conv_lp
280: ;-----------------------------------
281: ;     オクターブ設定
282: ;-----------------------------------
283: oct_ch:       mv   a,[x++]
284:                sub  a,30h
285: oct_ch_s:     cmp  a,10
286:                jpnc mml_conv_lp
287:                mv   [(!octerve_adr)],a
288:                add  a,0e0h
289:                mv   [y++],a
290:                jp   mml_conv_lp
291: ;-----------------------------------
292: oct_1up:      mv   (!oct_flg),[(!octerve_adr)]
293: ;-----------------------------------
294: oct_up:       mv   a,[(!octerve_adr)]
295:                inc  a
296:                jr   oct_ch_s
297: ;-----------------------------------
298: oct_1dw:      mv   (!oct_flg),[(!octerve_adr)]
299: ;-----------------------------------
300: oct_dw:       mv   a,[(!octerve_adr)]
301:                dec  a
302:                jr   oct_ch_s
303: ;-----------------------------------------------------------------
304: ;     データ
305: ;-----------------------------------------------------------------
306: ;              00 R     60 F    C0 B
307: ;              10 C     70 #F   D0 T    DF nn TEMPNN
308: ;              20 #C    80 G    E0 O    EE DOWN EF UP
309: ;              30 D     90 #G   F0 reserve
310: ;              40 #D    A0 A
311: ;              50 #E    B0 #A   FF END
312: ;-----------------------------------------------------------------
313: mml_data1:    db   0a0h                ;a
314:                db   0c0h                ;b
315:                db   010h                ;c
316:                db   030h                ;d
317:                db   050h                ;e
318:                db   060h                ;f
319:                db   080h                ;g
320: mml_data2:    db   0b0h                ;#a
321:                db   010h                ;#b
322:                db   020h                ;#c
323:                db   040h                ;#d
324:                db   050h                ;#e
325:                db   070h                ;#f
326:                db   090h                ;#g
327: ;-----------------------------------------------------------------
328: not_data:     db   0ffh
329: octerve:      db   3
330: length:       db   5
331: octerve2:     db   3
332: length2:      db   5
333: octerve3:     db   3
334: length3:      db   5
335: MML_data_adr: ds                       3
336: format_data:  db   1
337:                endl
338: ;-----------------------------------
339: ;     初期設定１
340: ;-----------------------------------
341:                jr   init2
342: init1:        mv   y,0
343:                mv   [part1+o_count],y  ; 音程カウンタ reset
344:                mv   [part2+o_count],y  ; 音程カウンタ reset
345:                mv   [part3+o_count],y  ; 音程カウンタ reset
346:                mv   a,2
347:                mv   [part1+oct],a      ; オクターブ初期化
348:                mv   [part2+oct],a      ; オクターブ初期化
349:                mv   [part3+oct],a      ; オクターブ初期化
350:                retf
351: ;-----------------------------------
352: ;     初期設定２
353: ;-----------------------------------
354: init2:        mv   i,0
355:                mv   [part1+onp],i      ; 音長カウンタ reset
356:                mv   [part2+onp],i      ; 音長カウンタ reset
357:                mv   [part3+onp],i      ; 音長カウンタ reset
358:                mv   y,[part1+data_s_adr]
359:                mv   [part1+data_adr],y ; データアドレスセット
360:                mv   y,[part2+data_s_adr]
361:                mv   [part2+data_adr],y ; データアドレスセット
362:                mv   y,[part3+data_s_adr]
363:                mv   [part3+data_adr],y ; データアドレスセット
364: ;-----------------------------------
365: ;     メインルーチン
366: ;-----------------------------------
367: main2:        mv   x,part1
368:                call mml
369:                mv   x,part2
370:                call mml
371:                mv   x,part3
372:                call mml                ; 各パートデータセット
373: ;
374:                mv   a,0
375:                mv   b,a
376:                mv   a,[part1+l_count]
377:                mv   (part_c),[part2+l_count]
378:                cmp (part_c),0
379:                jrz  main2_skip1
380:                cmp (part_c),a
381:                jrnc main2_skip1
382:                mv   a,(part_c)
383: main2_skip1:  mv   (part_c),[part3+l_count]
384:                cmp (part_c),0
385:                jrz  main2_skip2
386:                cmp (part_c),a
387:                jrnc main2_skip2
388:                mv   a,(part_c)
389: main2_skip2:  cmp a,0
390:                jrz  exit               ; a = 最短音長（０＝ＥＮＤ）
```

ちなみに私は，メニュー形式のマシン語ファイルSAVER＆LOADERを作りました．同サイズのソース・ファイルより速く送受信ができます．8月号のTIME-ATTACKERで9秒ちょいです．（＾＿＾；）
ただ，送信したファイルが元の1．2〜1．7倍にもなるので，もう少し改良するつもりです．
また，16EEh，16EFhでタイム・カウント，16E2h〜16E9hにパスワードが入っています（パスワードはXORFFHで格納）．

```
391:                mv  il,a
392: ;
393:                mv  a,[part1+l_count]
394:                sub a,il
395:                mv  [part1+l_count],a
396:                mv  a,[part2+l_count]
397:                sub a,il
398:                mv  [part2+l_count],a
399:                mv  a,[part3+l_count]
400:                sub a,il
401:                mv  [part3+l_count],a
402: ;                                          ; 最短分減算
403:                add i,i                     ; ×4
404:                mv  a,[temp]                ; テンポ（微調整）
405:                mv  x,0
406: main2_lp1:     add i,i
407:                rc
408:                shr a
409:                jmc main2_skip
410:                add x,i                     ; x = ba * il
411: main2_skip:    cmp a,0
412:                jmz main2_lp1
413:                mv  [count],x
414: ;
415:                mv  (part_c),0
416:                mv  y,count1
417:                mv  a,[part1+onp]
418:                inc a
419:                jrz p1_skip
420:                dec a
421:                jrz p1_skip
422:                mv  ba,[part1+o_count]
423:                mv  [y++],ba
424:                inc (part_c)
425: p1_skip:       mv  a,[part2+onp]
426:                inc a
427:                jrz p2_skip
428:                dec a
429:                jrz p2_skip
430:                mv  ba,[part2+o_count]
431:                mv  [y++],ba
432:                inc (part_c)
433: p2_skip:       mv  a,[part3+onp]
434:                inc a
435:                jrz p3_skip
436:                dec a
437:                jrz p3_skip
438:                mv  ba,[part3+o_count]
439:                mv  [y++],ba
440:                inc (part_c)
441: p3_skip:       mv  x,[count]
442:                mv  ba,[count1]
443:                mv  i,ba
444:                mv  y,i
445: ;-----------------------------------
446: ;      3 音処理設定
447: ;-----------------------------------
448:                mv  [!beep_out3!part1_co1+1],ba
449:                mv  [!beep_out3!part2_co2+1],i
450:                mv  [!beep_out3!part2_co1+1],i
451:                mv  [!beep_out3!part3_co1+1],y
452:                mv  [!beep_out3!part3_co2+1],y
453:                mv  [!beep_out3!part3_co3+1],y
454:                mv  [!beep_out3!part3_co4+1],y
455:                dec (part_c)
456:                jpz beep_out3
457:                mv  i ,[count2]
458: ;-----------------------------------
459: ;      2 音処理設定
460: ;-----------------------------------
461:                mv  [!beep_out3!part2_co2+1],i
462:                mv  [!beep_out3!part2_co1+1],i
463:                dec i
464:                dec i
465:                dec i
466:                dec (part_c)
467:                jpz beep_out3
468:                mv  ba ,[count3]
469: ;-----------------------------------
470: ;      単音処理設定
471: ;-----------------------------------
472:                mv  [!beep_out3!part1_co1+1],ba
473:                dec ba
474:                dec ba
475:                dec ba
476:                dec (part_c)
477:                jpz beep_out3
478: ;-----------------------------------
479: ;      無音処理設定
480: ;-----------------------------------
481:                jr  beep_out0
482: ;-----------------------------------
483: exit:          retf
484: ;-----------------------------------
485: ;      データ解析
486: ;-----------------------------------
487: ;              x = part parameter address
488: ;ret:          cy = 1  data end
489: ;              cy = 0i = 音程 [x+o_count]  テンポ [temp]
490: ;              a = 音長 [x+l_count]        音符 [x+onp]
491: ;y = 保存 :x = part paramater address      オクターブ [x+oct]
492: mml:           pushuy
493:                mv  a,[x+onp]
494:                cmp a,0ffh
495:                jrz mml_exit
496:                mv  a,[x+l_count]
497:                cmp a,0
498:                jrnz skip_exit
499:                mv  y,[x]                   ; ＭＭＬデータアドレス
500: mml_loop:      mv  a,[y++]
501:                cmp a,0ffh
502:                jrz mml_exit
503:                cmp a,0dfh
504:                jrz temp_ch
505:                cmp a,0e0h
506:                jrc mml_onp
507: ;-----------------------------------
508: oct_ch:        and a,0fh
509:                mv  [x+oct],a               ; オクターブのセット
510:                jr  mml_loop
511: ;-----------------------------------
512: temp_ch:       mv  a,[y++]
513:                mv  [temp],a                ; テンポのセット
514:                jr  mml_loop
515: ;-----------------------------------
516: mml_exit:      mv  [x+onp],a               ; 音符コード記憶
517:                mv  a,0
518:                mv  [x+l_count],a
519:                popu y
520:                sc
521:                ret
522: ;-----------------------------------
523: skip_exit:     mv  i,[x+o_count]
524:                popu y
525:                rc
526:                ret
527: ;-----------------------------------
528: mml_onp:       mv  [x],y                   ; ＭＭＬデータアドレス
(3byte)
529:                pushua
530:                and a,0f0h
531:                ror a                       ; ローテイトライト（÷2）
532:                ror a                       ; ローテイトライト（÷2）
533:                mv  il,a
534:                ror a                       ; ローテイトライト（÷2）
535:                ror a                       ; ローテイトライト（÷2）
536:                mv  [x+onp],a               ; 音符コード記憶
537:                add a,il                    ; 1+4
538:                add a,a                     ; ×2（a÷16×10）
539:                mv  y,mml_data              ; mml tone data
540:                add y,a
541:                mv  a,[x+oct]               ; オクターブ（O）
542:                add a,a                     ; ×2
543:                add y,a
544:                mv  i,[y]                   ; 音程カウンタset
545:                mv  [x+o_count],i           ; 音程カウンタset
546: ;-----------------
547:                popu a
548:                and a,0fh
549:                mv  y,length_data
550:                add y,a
551:                mv  a,[y]                   ; 音長カウンタ
552:                mv  [x+l_count],a
553:                popu y
554:                rc
555:                ret
556: ;-----------------------------------------------------------------
557: ;      音発生ルーチン 1loop = 37state 48.1uS
558: ;-----------------------------------------------------------------
559: ;-----------------------------------
560: ;      音発生（0音）
561: ;-----------------------------------
562: ;              x = 発声音長カウンタ    78state
563: beep_out0:     local
564: loop:          mv  i,4                     ;6  6*(a)+2
565: loop1:         dec i                       ;6
566:                jmz loop1                   ;4,6
567:                mv  (!beep),2h              ;8  ブザーＯＦＦ
568:                nop                         ;2
569:                nop                         ;2
570:                nop                         ;2
571: b_ent:         dec x                       ;6
572:                jmz loop                    ;4,6
573:                jp  !main2
574:                endl
575: ;-----------------------------------
576: ;      音発生（3音）
577: ;-----------------------------------
578: ;              ba= part1 カウンタ
579: ;              i = part2 カウンタ
580: ;              y = part3 カウンタ
581: ;              x = 発声音長カウンタ  78state
582: beep_out3:     local
583: ;              mv  [part1_co1+1],ba
584: ;              mv  [part2_co1+1],i
585: ;              mv  [part2_co2+1],i
586: ;              mv  [part3_co1+1],y
587: ;              mv  [part3_co2+1],y
588: ;              mv  [part3_co3+1],y
589: ;              mv  [part3_co4+1],y
590: loop:          dec ba                      ;6
591:                jmz skip1                   ;4,6
592: part1_co1:     mv  ba,0                    ;6
593:                mv  (!beep),2h              ;8  ブザーＯＦＦ
594:                dec i                       ;6
595:                jmz skip2                   ;4,6
596: part2_co1:     mv  i,0                     ;6
597:                dec y                       ;6
598:                jmz skip3                   ;4,6
599: part3_co1:     mv  y,0                     ;8
600:                mv  (!beep),12h             ;8  ブザーＯＮ
601:                dec x                       ;6
602:                jmz loop                    ;4,6
603:                jr  exit
604:
605: skip3:         nop                         ;2
606:                nop                         ;2
607:                nop                         ;2
608:                mv  (!beep),12h             ;8  ブザーＯＮ
609:                dec x                       ;6
610:                jmz loop                    ;4,6
611:                jr  exit
612:
613: skip2:         nop                         ;2
614:                nop                         ;2
615:                dec y                       ;6
616:                jmz skip3                   ;4,6
617: part3_co2:     mv  y,0                     ;8
618:                mv  (!beep),12h             ;8  ブザーＯＮ
619:                dec x                       ;6
620:                jmz loop                    ;4,6
621:                jr  exit
622:
623: skip1:         nop                         ;2
624:                nop                         ;2
625:                mv  (!beep),2h              ;8  ブザーＯＦＦ
626:                dec i                       ;6
627:                jmz skip12                  ;4,6
628: part2_co2:     mv  i,0                     ;6
629:                dec y                       ;6
630:                jmz skip3                   ;4,6
631: part3_co3:     mv  y,0                     ;8
632:                mv  (!beep),12h             ;8  ブザーＯＮ
633:                dec x                       ;6
634:                jmz loop                    ;4,6
635:                jr  exit
636:
637: skip12:        nop                         ;2
638:                nop                         ;2
639:                dec y                       ;6
640:                jmz skip13                  ;4,6
641: part3_co4:     mv  y,0                     ;8
642:                mv  (!beep),12h             ;8  ブザーＯＮ
643:                dec x                       ;6
644:                jmz loop                    ;4,6
645:                jr  exit
646:
647: skip13:        nop                         ;2
648:                nop                         ;2
649:                nop                         ;2
650:                nop                         ;2
651:                nop                         ;2
652:                nop                         ;2
653:                nop                         ;2
654: ;
655:                dec x                       ;6
656:                jmz loop                    ;4,6
657: exit:          mv  (!beep),2h              ;8  ブザーＯＦＦ
658:                jp  !main2
659:                endl
660: ;-----------------------------------------------------------------
661: ;      データ
662: ;-----------------------------------------------------------------
663: ;-----------------------------------
664: ;      音程データ
665: ;-----------------------------------
666: mml_data:      ;   O0   O1   O2   O3   O4
667:                dw  000h, 000h,  000h,  000h,  000h   ;R
668:                dw  12Eh, 097h,  04Ch,  026h,  013h   ;C
669:                dw  11Dh, 08Fh,  048h,  024h,  012h   ;#C
670:                dw  10Dh, 087h,  044h,  022h,  011h   ;D
671:                dw  0FEh, 07Fh,  040h,  020h,  010h   ;#D
672:                dw  0F0h, 078h,  03Ch,  01Eh,  00Fh   ;E
673:                dw  0E2h, 071h,  039h,  01Dh,  00Fh   ;F
674:                dw  0D6h, 06Bh,  036h,  01Bh,  00Eh   ;#F
675:                dw  0CAh, 065h,  033h,  01Ah,  00Dh   ;G
676:                dw  0BEh, 05Fh,  030h,  018h,  00Ch   ;#G
677:                dw  0B4h, 05Ah,  02Dh,  017h,  00Ch   ;A
678:                dw  0AAh, 055h,  02Bh,  016h,  00Bh   ;#A
679:                dw  0A0h, 050h,  028h,  014h,  00Ah   ;B
680: ;
681: length_data:   db  003h,006h,009h,00Ch,012h
682:                db  018h,024h,030h,048h,060h
683:                db  002h,004h,006h,010h,020h,001h
684: ;-----------------------------------------------------------------
685: temp:          db  1                       ; テンポ（コマンド内）
686: count:         dp  0                       ; 発声音長カウンタ(3byte)
687: count1:        db  0,0                     ; 音程カウンタ1
688: count2:        db  0,0                     ; 音程カウンタ2
689: count3:        db  0,0,0                   ; 音程カウンタ3
690: ;
691: part1:         ds  16                      ; 音長カウンタ
692: part2:         ds  16                      ; 音長カウンタ
693: part3:         ds  16                      ; 音長カウンタ
694: ;
695: r_flg:         ds  1                       ;
696: data_buff:     ds  1
697: ;
698: ;-----------------------------------------------------------------
699:                end
```

リスト3　サンプル・ミュージックBASICリスト

```
1000  CLEAR :CLS :WAIT 0
1010  LOCATE 0, 0:PRINT "PLAY3 サンプ ルミ
      ュージ ック":PRINT "COMPOSED BY KAZ
      UMI MATSUNOKI":PRINT "(C) 1993.
      9. 12"
1020  PLAY "T80O1:O1"
1030  *A:PRINT "*A";
1040  PLAY "L3>C<RL8R:O1L3CC#DL1CFL5
      FL1#F#FGG
1050  PLAY "O1L8RL5G:L3CC#DL1C<#AL5#
      AL1#AG#A>B
1060  PLAY "L9>C<:L3CC#DL1CFL5FL1#F#
      FGG
1070  PLAY "L3#AR#AR#A#AL1#AG#AB:L3C
      C#DL1C<#AL5#AL1#AG#A>B
1080  *B1:PRINT "*B1";
1090  PLAY "O1L3R>C<#AGL4#A>C<L3G:O1
      L3CC#DL1CFL5FL1#F#FGG
1100  PLAY "L3>#D#DL5DL4<#A>C<L3#A:L
      3CC#DL1C<#AL5#AL1#AG#A>B
1110  PLAY "L3R>C<#AGL4#A>C<L3G:L3CC
      #DL1CFL5FL1#F#FGG
1120  PLAY "L3FFFRGL1GGG#AR#A:L3CC#D
      L1C<#AL5#AL1#AG#A>B
1130  *B2:PRINT "*B2";
1140  PLAY "O1L3R>C<#AGL4#A>C<L3G:O1
      L3CC#DL1CFL5FL1#F#FGG
1150  PLAY "L3>#D#DL5D<L4#A>C<L3#A:L
      3CC#DL1C<#AL5#AL1#AG#A>B
1160  PLAY "L3R>C<#AGL4#A>C<L3G:L3CC
      #DL1CFL5FL1#F#FGG
1170  PLAY "O2L1#DL3CL1#DL3CL1FL3CL1
      #FL3CL1GFG#A:L1CL3#DL1CL3#DL1C
      L3<#A>L1<GL3#AL1GG#A>D
1180  *C1:PRINT "*C1";
1190  PLAY "O2L9>C<:O1L3CC#DL1CFL5FL
      1#F#FGG
1200  PLAY "L3#AR#GRGRFR:L3CC#DL1C<#
      AL5#AL1#AG#A>B
1210  PLAY "L1#DFGL3C>C<L1G#AG#AL3>C
      C<L1G:L3CC#DL1CFL5FL1#F#FGG
1220  PLAY "L1F#DF#DF#DF#DF#DFL3G#AL
      1G:L3CC#DL1C<#AL5#AL1#AG#A>B
1230  *C2:PRINT "*C2";
1240  PLAY "O3L9G:L3CC#DL1CFL5FL1#F#
      FGG
1250  PLAY "L3FR#DRDR<#A>R:L3CC#DL1C
      <#AL5#AL1#AG#A>B
1260  PLAY "O2L1#A#AB>C<L3RL1#A#AB>C
      <L3RL1#A#AB>C<:L3CC#DL1CFL5FL1
      #F#FGG
1270  PLAY "L1RGGRGRGRGRFFF#AR#A:L1C
      L3#DL1CL3#DL1CL3<#AL1GL3#AL1GG
      #A>D
1280  GOTO *A
```

PS　ちなみに私は16E3hから4バイトにオート・パワーオフ用のプログラムを入れて重宝してます．　明日からテストだ！（Qせんじ）